Durable Execution Around a Sampler

Temporal for Physics-Checked Inference

Arun Sharma, University of Minnesota, Twin Cities

A Worker died 3 seconds in

> A test script ended the Worker with SIGKILL.
>
> The replacement Worker was ready in about 5 seconds.
>
> The next attempt still waited for a timeout.

Mac CPU, demo-scale model, five runs

Four decisions set what failure costs

- Retry: The Retry Policy selects another attempt.
- Stop: A failure can end further attempts.
- Notice: A timeout can end a silent attempt.
- Replay: Saved histories test changes to Workflow code.

What this is, and what it is not

- Every run used one Apple M4 Pro laptop CPU.
- The demo-scale model uses a 64 by 64 tile.
- The model trained briefly on synthetic data.
- The Temporal dev server used a local database file.
- Each scenario has five runs.

This is a research system, not a commercial production deployment.

Mac CPU, demo-scale model, five runs

A sampler is a long, checkable call

1500

sampler steps in the standard request

4

ensemble members in the standard request

16.130

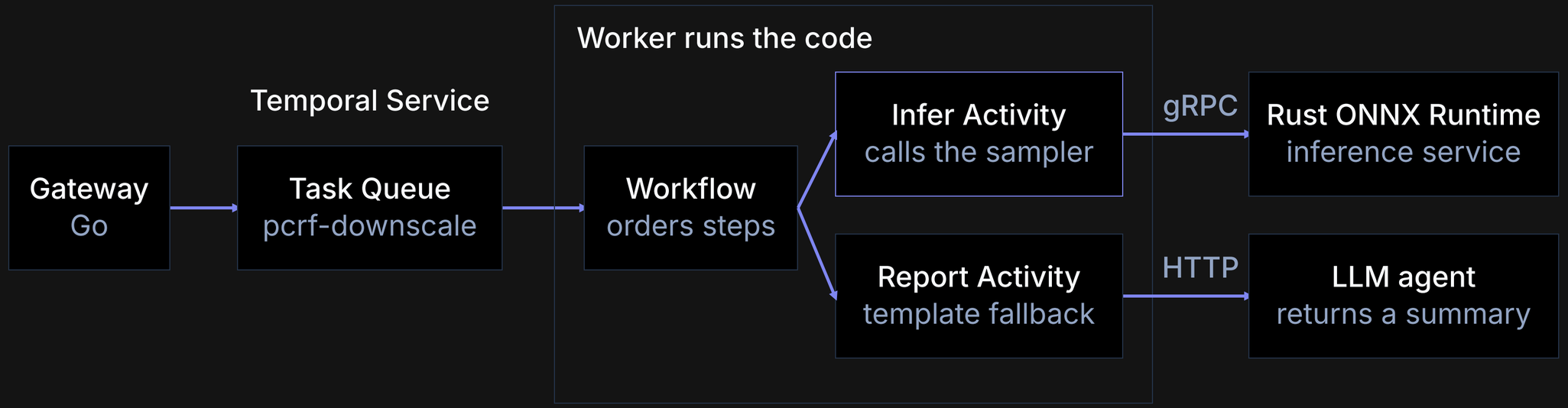
median seconds for a baseline call

The physics gate checks the ensemble mean only.

- NVR is the negative-value rate, and its limit is 0.
- MCE is the mass conservation error, and its limit is 1e-3.

Mac CPU, demo-scale model, five runs

One request, two Activities



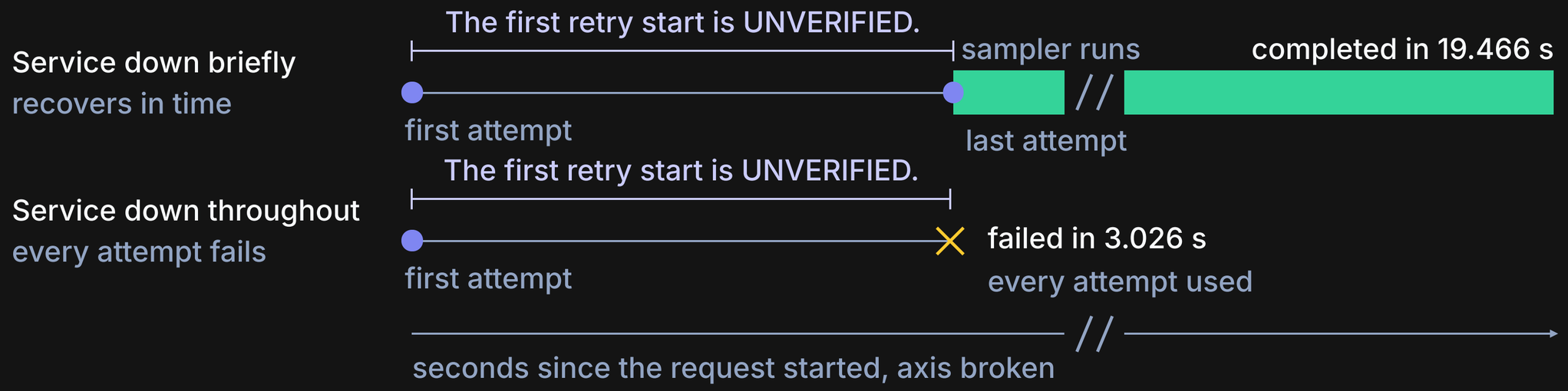
One request went through the gateway. The campaign started Workflows with the Temporal CLI.

Two Activities, one difference

```
options := workflow.ActivityOptions{
    StartToCloseTimeout: 2 * time.Minute,
    RetryPolicy:         &temporal.RetryPolicy{MaximumAttempts: 3},
}
reportCtx := workflow.WithActivityOptions(ctx, options)
if !job.DisableHeartbeat {
    options.HeartbeatTimeout = 10 * time.Second
}
inferCtx := workflow.WithActivityOptions(ctx, options)
```

Only the Infer Activity gets a Heartbeat Timeout. The Report Activity has none.

Same Retry Policy, two outcomes



Both lanes allow at most 3 attempts. The first retry start time is UNVERIFIED.

Mac CPU, demo-scale model, five runs

A physics gate decides what to retry

```
maxNVR, maxMCE := float32(0), float32(1e-3)
...
if math.IsNaN(float64(nvr)) || math.IsInf(float64(nvr), 0) ||
    math.IsNaN(float64(maxNVR)) || math.IsInf(float64(maxNVR), 0) ||
    maxNVR < 0 || nvr < 0 || nvr > maxNVR {
    failed = append(failed, "NVR <= max_nvr")
}
if math.IsNaN(float64(mce)) || math.IsInf(float64(mce), 0) ||
    math.IsNaN(float64(maxMCE)) || math.IsInf(float64(maxMCE), 0) ||
    maxMCE <= 0 || mce < 0 || mce >= maxMCE {
    failed = append(failed, "MCE < max_mce")
}
```

The code below this panel returns a Non-Retryable Error named PhysicsCheckFailed. The gate checks the ensemble mean only.

Invalid output stops after one attempt

> All five runs of a 20-step request failed the physics gate.
>
> Each run stopped after one attempt, in a median of 0.224 seconds.
>
> Physics check failed for NVR <= max_nvr, MCE < max_mce.
> NVR=0.107666016, MCE=4.189112, max_nvr=0, max_mce=0.001.

The gate checks the ensemble mean only.

Mac CPU, demo-scale model, five runs

The seed travels with the request

0.214

divergence error

0.108

negative-value rate

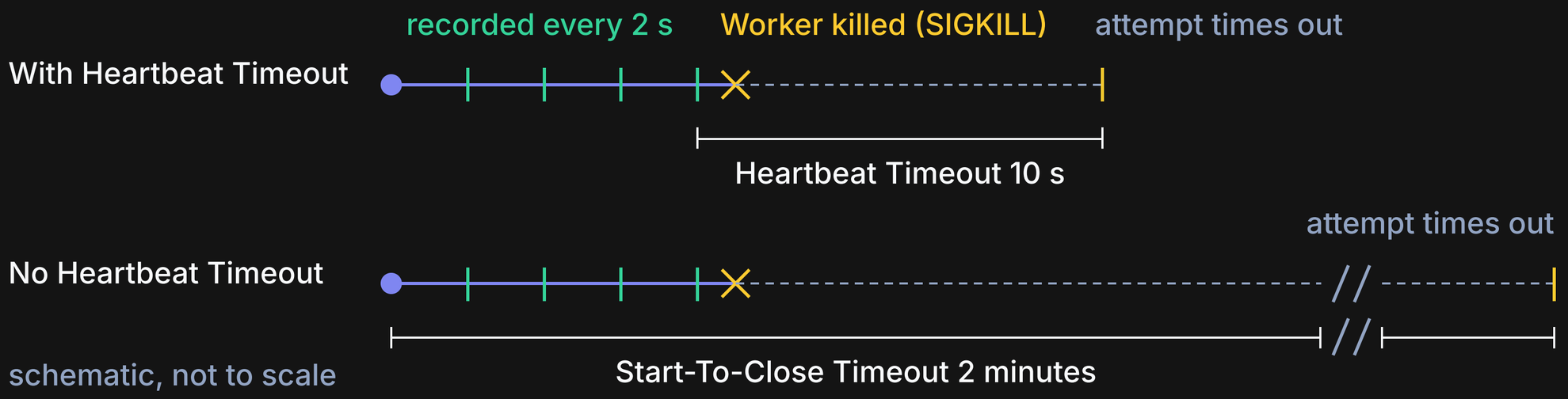
4.19

mass conservation error

- Both calls returned the same values.
- The same seed gave matching ensemble metrics on this CPU.
- Full field identity and other hardware are UNVERIFIED.

Mac CPU, demo-scale model, five runs

A dead Worker does not announce itself



The test ended the Worker with SIGKILL. A killed Worker sends no message.

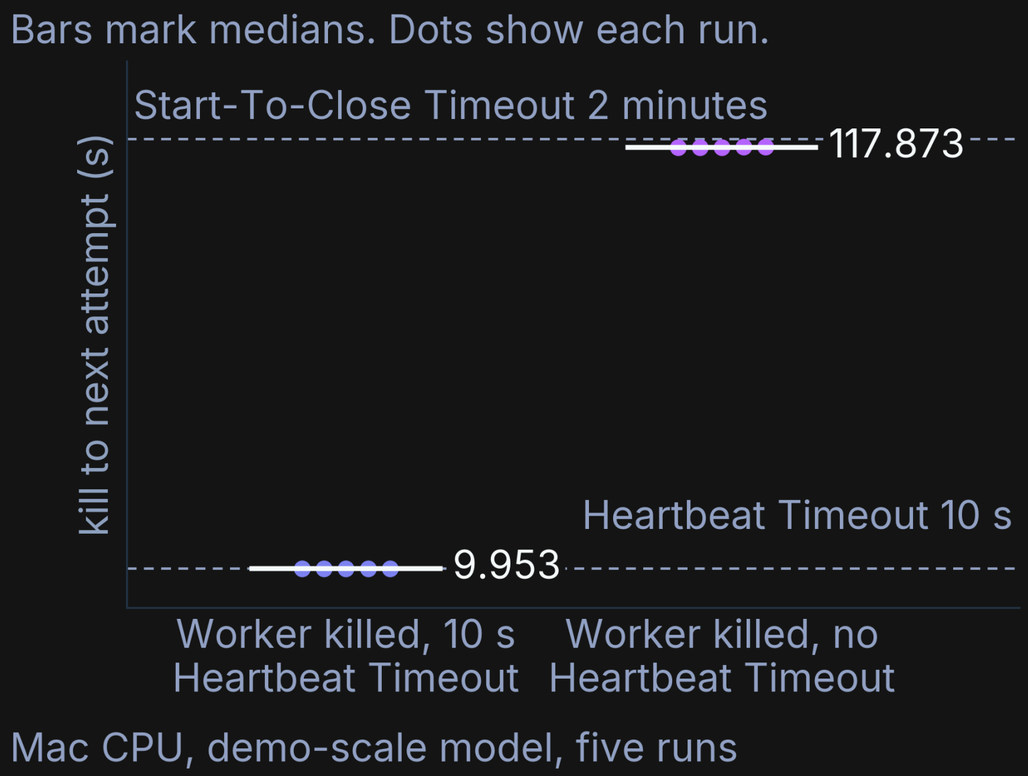
Heartbeat from inside the RPC wait

```
    ticker := time.NewTicker(2 * time.Second)
    defer ticker.Stop()
    var resp *pb.DownscaleResponse
waitForRPC:
    for {
        select {
        case out := <-done:
            resp, err = out.response, out.err
            break waitForRPC
        case <-ticker.C:
            activity.RecordHeartbeat(ctx, p.Seed)
```

The panel shows the start of the wait loop. The loop closes a few lines below it, and the Heartbeat records only the seed.

9.953 seconds against 117.873 seconds

- Both groups record Heartbeats.
- Only the timeout setting differs.
- The test ended the Worker with SIGKILL.



Both medians are consistent with the timeout settings. The interval includes retry backoff, Worker availability and dispatch delay.

Mac CPU, demo-scale model, five runs

The Event History names the timeout

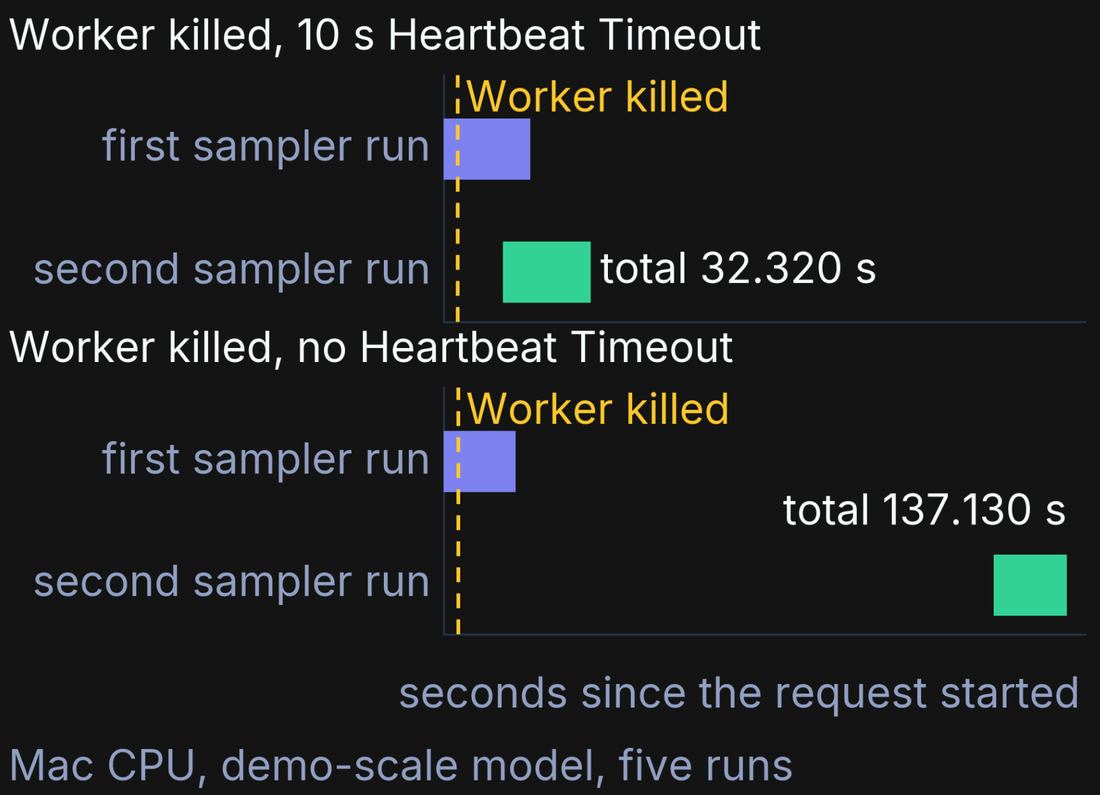
Event 6	Worker killed, 10 s Heartbeat Timeout	Worker killed, no Heartbeat Timeout
lastFailure message	activity Heartbeat timeout	activity StartToClose timeout
timeoutType	TIMEOUT_TYPE_HEARTBEAT	TIMEOUT_TYPE_START_TO_CLOSE
source	Server	Server

Event 6 of each Worker-killed Event History names the timeout that ended the first attempt.

Mac CPU, demo-scale model, five runs

Every recovery ran the sampler twice

- Recovery ran the sampler twice.
- This held in all 15 worker-kill runs.
- The test ended the Worker with SIGKILL.



Mac CPU, demo-scale model, five runs

A Heartbeat Timeout saves waiting time and does not save compute.

Mac CPU, demo-scale model, five runs

Replay a saved history before shipping

```
func TestReplayR2History(t *testing.T) {
    replayer := worker.NewWorkflowReplayer()
    replayer.RegisterWorkflow(DownscaleWorkflow)
```

The next line replays the saved Event History through the Replayer. The replay test covers one code transition and one saved history.

- A Replayer runs a saved history against new code.
- It fails when the commands no longer match.

An inserted timer fails the replay

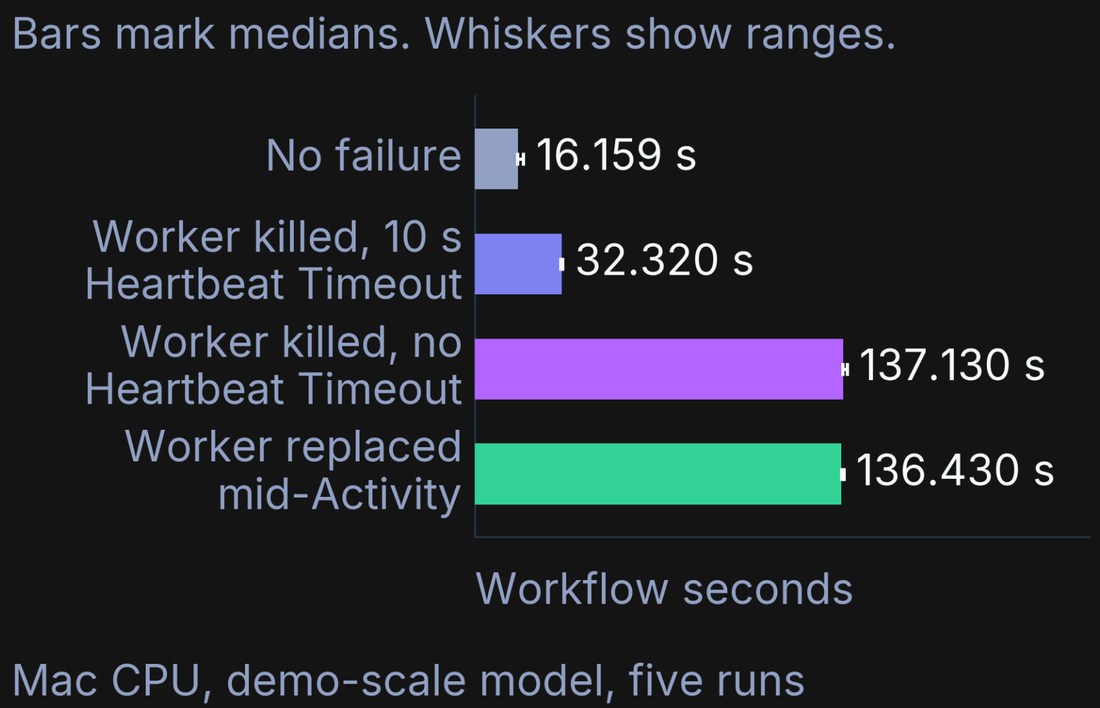
> A deliberate edit inserted a timer before Infer.
>
> The saved history schedules Infer at event 5.
>
> [TMPRL1100] lookup failed for scheduledEventID to activityID: scheduleEventID: 5, activityID: 5

The replay test covers one code transition and one saved history.

Mac CPU, demo-scale model, five runs

New code, old Activity options

- A newer Worker build took over.
- The old Heartbeat Timeout of 0 s stayed.
- The median wait to retry was 117.873 seconds.



The 117.873 seconds is consistent with the 2-minute Start-To-Close Timeout. The replay test covers one code transition and one saved history.

Mac CPU, demo-scale model, five runs

The LLM report is optional

```
    if a.AgentURL != "" {
        if txt, err := callAgentReport(ctx, a.AgentURL, r); err == nil {
            return txt, nil
        }
    }
    return fmt.Sprintf(
        "Downscaled a %dx%d field on %s in %.0f ms. Physics validity: "+
            "DE=%.4f, NVR=%.4f, MCE=%.4f.",
        r.Height, r.Width, r.Backend, r.LatencyMs, r.DE, r.NVR, r.MCE), nil
}
```

This is the body of the Report Activity.

- All five runs completed with the template report.
- Only a mocked test reached the unavailable branch.

Mac CPU, demo-scale model, five runs

What this does not show

- Every run used the Temporal dev server.
- The replay test covers one saved history.
- The physics gate checks the ensemble mean only.
- Cancellation ran only against a fake service.
- I have not run this in production.

This is a research system, not a commercial production deployment.

Four checks before you ship

- Retry: Domain checks identify Non-Retryable Errors.
- Stop: Maximum Attempts bounds the attempt count.
- Notice: A Heartbeat Timeout needs a plan for repeated work.
- Replay: Saved Event Histories test Workflow changes.

What I would test next

I would test real traffic, graceful shutdown and sampler cancellation.

Arun Sharma, University of Minnesota, Twin Cities

Backup slides

Scenario names and their codes

Scenario	Code	Group key
No failure	baseline	baseline
Worker killed, 10 s Heartbeat Timeout	F1	f1-hb
Worker killed, no Heartbeat Timeout	F1	f1-nohb
Service down briefly	F2	f2
Service down throughout	F3	f3
Invalid physics output	F4	f4
LLM agent unavailable	F5	f5
Worker replaced mid-Activity	F6	f6

All eight groups, medians and ranges

Group	Median (s)	Range (s)
baseline	16.159	15.767 to 18.080
f1-hb	32.320	31.829 to 32.483
f1-nohb	137.130	136.808 to 138.523
f2	19.466	19.347 to 20.541
f3	3.026	3.025 to 3.026
f4	0.224	0.222 to 0.232
f5	16.271	15.926 to 16.437
f6	136.430	136.338 to 137.164

Mac CPU, demo-scale model, five runs

The ten runs behind the headline

Run with Heartbeat Timeout	Seconds	Run without it	Seconds
f1-hb-1	9.973	f1-nohb-1	117.958
f1-hb-2	9.961	f1-nohb-2	117.873
f1-hb-3	9.926	f1-nohb-3	117.899
f1-hb-4	9.953	f1-nohb-4	117.860
f1-hb-5	9.926	f1-nohb-5	117.850

Each value runs from the kill to the next attempt. The test ended the Worker with SIGKILL. The interval includes retry backoff, Worker availability and dispatch delay.

Mac CPU, demo-scale model, five runs

Recorded timeouts and Retry Policy

Setting	Infer	Report
Start-To-Close Timeout	2 minutes	2 minutes
Heartbeat Timeout	10 s	none
Maximum attempts	3	3
First retry interval	1 s	1 s
Backoff coefficient	2	2
Maximum interval	100 s	100 s

The environment and the tests

Item	Recorded value
Machine	Apple M4 Pro, 14 cores
Model	64 by 64 tile, base width 32
Temporal	CLI 1.7.2, Server 1.31.1
Go and Rust	Go 1.26.4, Rust 1.96.0
Go tests	10 top-level tests passed
Rust tests	6 passed on the private build
Remote CI	UNVERIFIED

What went wrong during the campaign

- The Temporal CLI inherited settings that did not match the local server.
- Flags now disable inherited settings and select the local server.
- The first detached launch exited silently in the execution tool.
- A restart interrupted a baseline run, and the runner recorded it separately.
- A replacement run used a new Workflow Id.

The evidence trail and how to rerun it

- Commit 9aa337d is on a local, unpushed branch.
- The runbook is experiments/temporal/RUNBOOK.md.
- The runner saves each run, then starts another.
- An audit script checked 40 runs and their logs.

This link opens PC-RF. The Temporal layer is not pushed yet.

`github.com/arunshar/pc-rf`

Mac CPU, demo-scale model, five runs